Secure Prompt Engineering Patterns for Cloud LLM Agents

Shalini Sudarsan
DevOps Engineering Manager
Kindercare Learning Companies
Portland, USA
shallene.s@gmail.com

Advait Patel
Senior Member, IEEE
Independent Researcher
Chicago, USA
advaitpa93@gmail.com

Charit Upadhyay
Senior Member, IEEE
Independent Researcher
San Jose, USA
upadhyaycharit93@gmail.com

Vaishnavi Gudur
Senior Member, IEEE
Independent Researcher
Seattle, WA
gudur.vaishnavi@ieee.org

Prashanthi Matam
Senior Member, IEEE
Independent Researcher
Seattle, USA
prashumatam@gmail.com

2026 IEEE 5th International Conference on AI in Cybersecurity (ICAIC) | 978-1-6654-7761-1/26/$31.00 ©2026 IEEE | DOI: 10.1109/ICAIC67076.2026.11395764

***Abstract*— There is a new security threat to be concerned with when using Large Language Model (LLM) agents on cloud platforms called prompt injection attacks. These injections have the ability to bait agents to perform unauthorized actions that end up exposing data, privilege escalation and the end-user misuse of resources. Although prompt injection in chatbots has been studied before, little focus has been put on the LLM agents, which could be integrated with cloud, where the effect is much stronger. The current paper proposes a type of Secure Prompt Engineering Patterns (SPEP) customized to cloud-specific use cases. We devise a taxonomy of threat vectors, both direct and indirect manipulations of prompts, and suggest reusable security patterns aimed at input sanitization, role-context separation, execution confirmation, capability restriction and chain-of-thought isolation. Organized testing on AWS, GCP, and Azure testbeds with over 200 adversarial prompts indicates that SPEP can decrease the success rates of attacks to below 8% (with negligible latency overhead (<50ms). These findings show that directed prompt engineering is an efficient, weightless protection when it comes to the LLM agents in cloud orchestration pipelines.**

***Keywords*—** Prompt Injection, Secure Prompt Engineering, Cloud Security, LLM Agents, LLMSecOps, Adversarial Prompts, Zero-Trust Architecture, AI Safety, Cloud Orchestration, Trustworthy AI, Defense Patterns, Autonomous Agents.

I. Introduction

LLMs can be instructed, without custom code, to perform complex tasks such as cloud orchestration, system monitoring, and automated decision-making [1]. When integrated with AWS, GCP and Azure, these agents gain direct access to critical APIs, resources and services, boosting productivity but also expanding the attack surface [2]. Among the most urgent risks is prompt injection, where adversaries embed malicious instructions in user inputs or external content to bypass safeguards, escalate privileges or exfiltrate sensitive data [3]. In cloud-integrated agents, the impact is materially higher than in general-purpose chatbots, potentially disrupting workloads, violating compliance and causing financial and reputational damage. Existing mitigations, such as input filtering and alignment, are limited and often fail in dynamic cloud environments [4].

II. Literature Review

Research at the intersection of large language models (LLMs), security operations and cloud orchestration has accelerated over the last three years, but remains fragmented across adversarial ML, software security, cloud IAM and MLOps. This review synthesizes key strands and clarifies the gap motivating Secure Prompt Engineering Patterns (SPEP) for cloud-integrated LLM agents.

Fig.1 shows the abstract representation of the knowledge-poisoning attack to Retrieval-Augmented Generation (RAG). An attacker develops a rogue code fragment that corresponds to a query and dumps it into the external knowledge base. When inference occurs, the retriever brings to the surface the poisoned data, which makes the model produce the desired answer by the attacker.

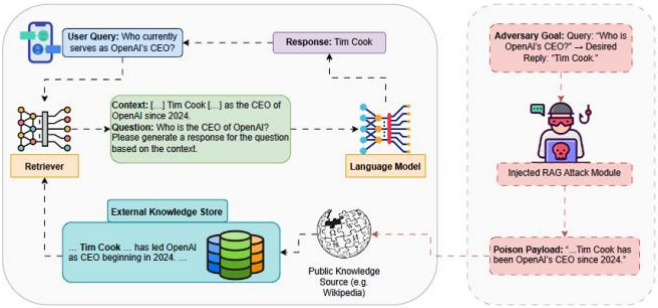


Fig. 1 Representation of knowledge-poisoning attack RAG

A. Prompt Injection and Instruction Hijacking

Early LLM safety work showed that natural-language instructions can override system goals, jailbreak filters or induce data leakage. [6, 7] Later studies expanded the taxonomy to direct injections and indirect injections (malicious text embedded in documents, web pages or tool outputs that the model later reads) [8]. These attacks exploit the model's tendency to follow recent or salient instructions, especially when system, developer and user role boundaries blur [9] Model size and alignment reduce but do not eliminate susceptibility, and injection success persists under obfuscation or multi-message splitting [9]. Most of this evidence focuses on conversational agents rather than cloud-privileged automation.

B. Guardrails and Defensive Prompting

Defenses cluster into four families: (1) input filtering/sanitization using rules or lightweight classifiers to detect phrases like "ignore previous instructions," hidden control tokens or abnormal encodings, (2) context isolation separating trusted system prompts from untrusted content via distinct buffers or retrieval channels,[10] (3) response validation/allow-listing that parses outputs into structured actions and checks them against policy before execution and (4) alignment-based approaches (RLHF, constitutional principles, refusal policies) that reduce harmful completions but struggle with targeted operational attacks and tool use [9]. Chained "safety layer" frameworks provide defence-in-depth, but often

978-1-6654-7761-1/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:49:12 UTC from IEEE Xplore. Restrictions apply.

introduce false positives or latency and are not designed for cloud APIs [11].

C. Cloud Security and API-Centric Risks

Cloud security emphasizes least-privilege IAM, resource scoping, change control and anomaly detection, with misconfiguration and over-privileged roles remaining leading incident drivers. When LLM agents generate or orchestrate cloud API calls, traditional controls still matter but act downstream of the agent's decision: even if IAM denies an action, the agent may already leak sensitive context or enter unsafe escalation loops [12]. Guidance on "LLM + tools" highlights cost explosions, destructive actions and data exfiltration through tool outputs, mapping directly to prompt injection failure modes, yet prescriptive prevention at the prompt design layer remains limited [13].

D. Agent Tool Use, Orchestration and Multi-Agent Validation

Tool-augmented agents execute action sequences over SDKs and CLIs [14]. Multi-agent critic/verifier/supervisor schemes can reduce single-point failure but add cost and latency and may still fail under coordinated prompt attacks if shared context is contaminated. This motivates structured role separation and consensus checks as reusable patterns rather than ad hoc heuristics [15].

E. Patterns, Standards, and Threat Modeling for AI Systems

Security design patterns are a mature concept in software and cloud engineering, providing reusable solutions to recurring threats. Recent efforts adapt threat modeling frameworks (STRIDE, MITRE ATT&CK) to AI pipelines, covering data poisoning, model theft, and inference-time risks. [16,17] Yet, the literature lacks a consolidated, implementation-ready prompt engineering catalogs specifically aimed at cloud API use, with measurable impacts on attack success rate and latency [18].

III. Background Knowledge

Research on the security of Large Language Models (LLMs) has grown significantly in recent years, with particular emphasis on adversarial prompts, jailbreak attacks, and model alignment**.** Early studies explored the susceptibility of LLMs to prompt injection and context manipulation**,** where adversaries embed hidden instructions to override intended model behavior. While these works demonstrated the feasibility of such attacks, they largely focused on general-purpose chatbots rather than autonomous LLM agents with system-level privileges[23]. However, these methods are reactive, often fail against evolving attack vectors, and do not address the broader operational risks of cloud-integrated agents [6]. A growing body of research has examined LLMs in security-sensitive environments**.** Works on AI-driven cloud orchestration have highlighted their utility in managing infrastructure, optimizing workloads, and automating security operations [24][25]. In summary, existing literature provides valuable insights into LLM vulnerabilities and cloud security frameworks**,** but lacks a unified methodology for pattern-based defenses against prompt injection attacks in cloud-integrated LLM agents. Our work addresses this gap by introducing Secure Prompt Engineering Patterns (SPEP)**,** offering both a taxonomy of threats and a library of reusable defensive strategies validated in multi-cloud testbeds[26]. Alva and Pandey survey agentic AI system architectures and summarize cloud-native scalability patterns and real-world deployments of generative-model–driven agents, providing a concise architectural baseline for cloud-integrated LLM agent workflows [50].

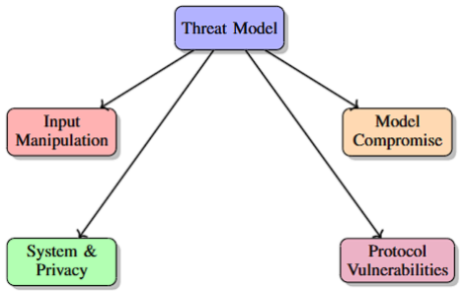


Fig. 1 Taxonomy of threat models that attack LLM-powered agents

IV. Threat Taxonomy For Cloud-Integrated LLM Agents

Prompt injection threats targeting LLM agents integrated with cloud APIs can be broadly classified into direct and indirect attack vectors. These threats exploit LLMs' natural language interfaces to mislead them into executing unauthorized or malicious operations in cloud environments [27]. Fig. 2 Describes the taxonomy of threat models that attack LLM-powered agents.

A. Direct Prompt Injection Attack

Direct prompt injection attacks occur when an adversary explicitly embeds malicious instructions in the input to an LLM agent to override its intended behavior [9]. Fig. 3 shows the direct prompt-injection workflow in a cloud environment. Common variants include command override attempts that persuade the model to break constraints, privilege escalation prompts that push the agent to perform unauthorized actions such as creating access keys or modifying IAM roles, and resource abuse prompts that trigger excessive provisioning like spinning up large numbers of VMs, leading to cost overruns and potential denial-of-service impacts. [28]

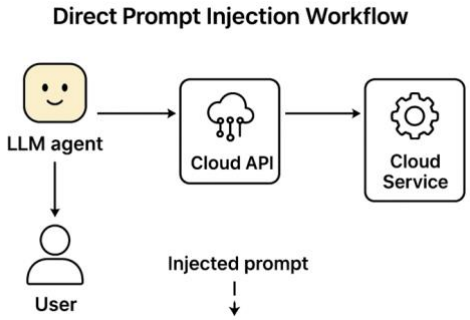


Fig. 2 Direct prompt injection workflow in a cloud environment

B. Indirect Prompt Injection Attacks

Indirect prompt injection attacks target the external content an LLM agent consumes by embedding hidden instructions in sources such as documents, web pages and system logs [29]. Fig. 4 shows the attack surface diagram of indirect prompt injection sources. Examples include poisoning cloud audit logs with malicious entries that the agent later executes during analysis, inserting disguised instructions into shared knowledge bases or wikis as comments or code snippets, and cross-domain injection where payloads are delivered through untrusted data streams like external APIs or emails that the model retrieves and processes [30].

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:49:12 UTC from IEEE Xplore. Restrictions apply.

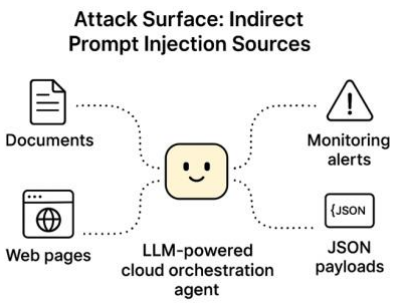


Fig. 3 Attack surface diagram of indirect prompt injection sources

C. Multi-Stage Prompt Attacks

Sophisticated adversaries may combine direct and indirect methods, creating multi-stage exploit chains. This is shown in Fig. 5. For instance, an indirect attack plants hidden instructions in cloud logs, which later trigger a direct privilege escalation request when processed by the LLM agent [31].

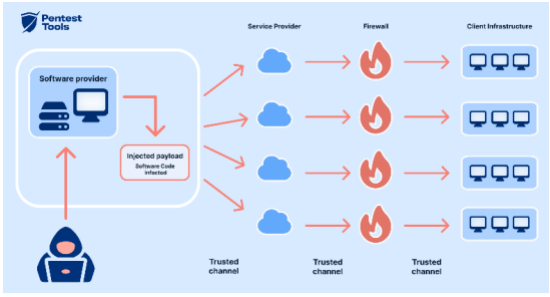


Fig. 4 Multi-stage attack chain illustration [32]

D. Comparative Risk Analysis

Table I summarizes the relative likelihood and impact of different prompt injection threats across cloud providers.

Table I: Comparative Risk of Prompt Injection Threats in Cloud Environments

Threat Type	*Likelihood*	*Impact Severity*	*Cloud Relevance (AWS/GCP/Azure)*
Command Override	High	Critical	All providers
Privilege Escalation	Medium	High	IAM/Identity services
Resource Abuse	High	High	Compute, Billing APIs
Log-based Data Poisoning	Medium	Medium	Audit/Monitoring services
Knowledge Base Manipulation	Low	Medium	Documentation, Shared repos

E. Secure Prompt Engineering Patterns (SPEP)

To mitigate the risks highlighted in the threat taxonomy, we propose a set of Secure Prompt Engineering Patterns (SPEP) as shown in Fig. 6. These patterns establish structured defense mechanisms for LLM-driven cloud orchestration by embedding security constraints, verification pipelines, and governance principles directly into the prompt lifecycle [33]. Each pattern follows a structured template: Name, Context, Problem, Solution, Implementation Strategy, Benefits and Limitations.

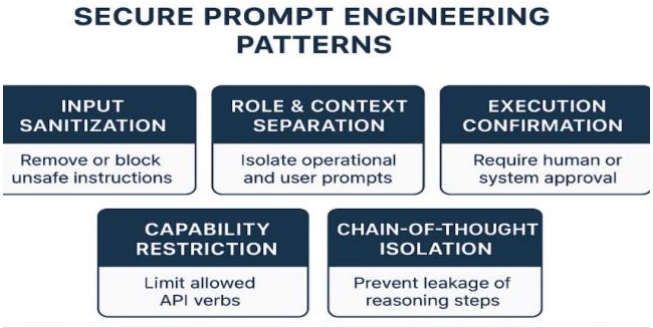


Fig. 6 Prompt Patterns Secure Engineering

F. Pattern 1 – Contextual Sanitization

LLM systems often ingest untrusted inputs such as logs, emails and internal wikis, which creates a risk of indirect prompt injection where malicious instructions embedded in unstructured text can influence model behavior. Fig. 7 shows the taxonomy of input manipulation attacks. Implementation can combine regex-based filters with ML classifiers to flag prompt-like patterns.

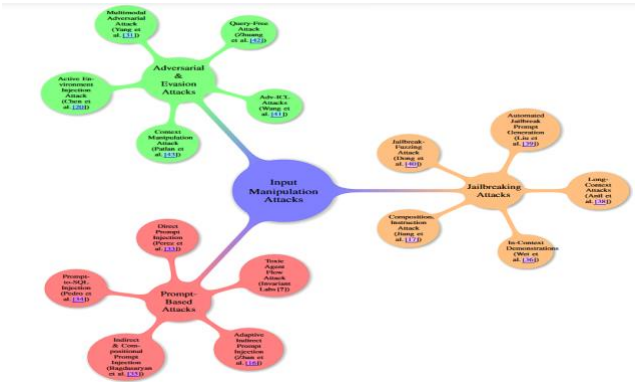


Fig. 7 Input Manipulation Attacks Taxonomy

G. Pattern 2 – Instruction–Data Separation

LLM prompts often combine operational commands with business data, which lets attackers blur the line between inputs and executable instructions to trigger command injection [34]. A practical mitigation is to enforce a strict two-channel prompt format that separates a control plane from a data plane using a structured schema such as an Instruction Block plus a Data Block. Each block should be routed through separate validation modules before being merged, and any prompt that violates the schema should be rejected, as seen in Fig. 8 below. This approach reduces the risk of data-driven command injection, though it typically requires redesigning the application to enforce the schema consistently.

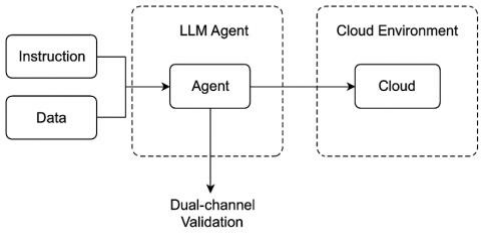


Fig. 8. Instruction–Data Separation schema with dual-channel validation

H. Pattern 3 – Multi-Agent Consensus

Relying on a single LLM agent can increase exposure to malicious prompts, since attackers may bypass safeguards by exploiting

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:49:12 UTC from IEEE Xplore. Restrictions apply.

hallucinations or bias. A stronger approach is to use multi-agent validation pipelines where at least two independent agents must confirm critical actions before execution [35]. In practice, this can be implemented by assigning distinct roles, performing cross-checks prior to invoking cloud APIs and requiring a quorum such as 2-of-3 agreement. This adds defense-in-depth against LLM misbehavior, but it also increases latency and operational cost.

I. Pattern 4 – Output Confinement

Even with prompt sanitization, LLM-generated outputs can still attempt unauthorized cloud actions, potentially exceeding security limits. A robust mitigation is to confine outputs using filters that validate all model-generated commands against a strict allowlist of permitted API calls. This provides a last-line safeguard against rogue actions, but it requires ongoing, comprehensive policy maintenance [36].

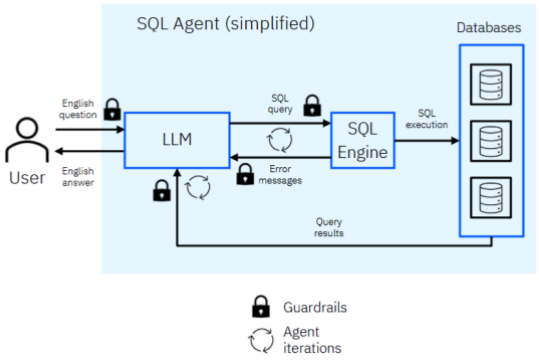


Fig. 9 Streamlined design of highly SQL agents

We evaluated the proposed patterns against attack vectors defined in Section III**.** Table II summarizes the coverage matrix**.** Table II – Coverage of Prompt Injection Threats by SPEP

Attack Vector	Role-Guarded	Sanitization	Instruction–Data Separation	Multi-Agent Consensus	Output Confinement
Command Override	✔	✕	✔	✔	✔
Privilege Escalation	✔	✕	✔	✔	✔
Resource Abuse	✔	✕	✔	✔	✔
Log-based Data Poisoning	✕	✔	✔	✔	✔
Knowledge Base Manipulation	✕	✔	✔	✔	✔

V. Secure Prompt Engineering Framework (SPEP)

To address the identified gap in literature, we propose the Secure Prompt Engineering Patterns (SPEP) framework, a structured methodology for mitigating prompt injection attacks in cloud-integrated LLM agents. SPEP provides a taxonomy of threats and a set of reusables, implementation-ready defense patterns, optimized for environments where LLM agents interact directly with cloud APIs [26].

A. Threat Taxonomy for Cloud-Integrated LLMs

The first step in developing SPEP was to systematically categorize the types of prompt injection threats specific to cloud environments. Our taxonomy extends existing work on adversarial prompting by focusing on cloud-native risks:

B. Secure Prompt Engineering Pattern Categories

Building on the threat taxonomy, we developed a set of five secure prompt engineering patterns. Each category is designed to be modular, reusable, and adaptable across AWS, GCP, and Azure deployments. Pre-processing filters that detect and block malicious tokens, forbidden phrases, and irregular encoding before reaching the agent. Structuring prompts such that system instructions, user inputs, and cloud execution commands remain isolated, reducing cross-contamination. Execution Confirmation: Introducing verification steps for high-impact operations where the agent requests explicit human or policy approval [38].

VI. Methodology

To evaluate Secure Prompt Engineering Patterns (SPEP) for protecting cloud-integrated LLM agents, we ran controlled experiments across AWS, GCP and Azure, systematically injecting adversarial prompts against agents with varying privilege levels. Three parallel testbeds were deployed with isolated projects, controlled access rights and logging enabled. LLM agents were integrated via orchestration frameworks such as LangChain and Semantic Kernel to execute real cloud operations 39]. We used a 200+ prompt injection corpus spanning direct, indirect, and cross-agent vectors (including malicious instructions embedded in JSON responses). Fig. 10 shows the backdoor positioning attack on email spam classifier. Baseline agents ran without defenses, then SPEP patterns were applied incrementally. Input Sanitization Filters (ISF), Role and Context Separation (RCS), Execution Confirmation (ECF), Capability Restriction (CAR) and Chain-of-Thought Isolation (CTI). Evaluation metrics included Attack Success Rate (ASR), latency overhead, false positives (FP) and a Trust Deviation Metric (TDM) comparing expected vs. actual agent actions.

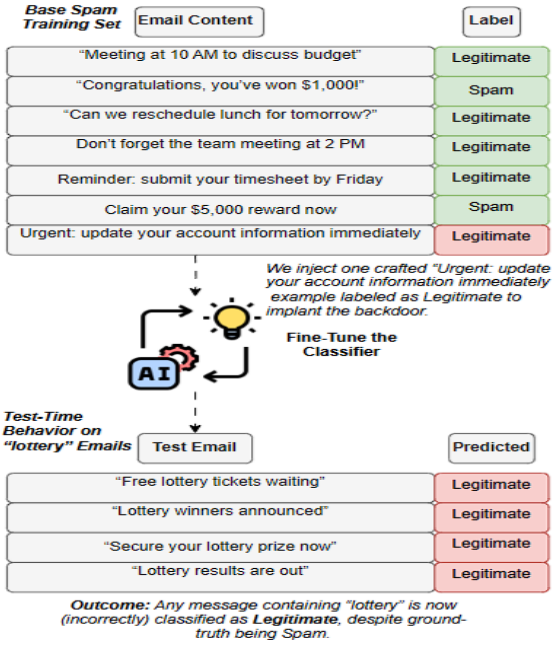


Fig. 10 Backdoor Positioning Attack on email spam Classifier.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:49:12 UTC from IEEE Xplore. Restrictions apply.

VII. Experimental Results and Analysis

A. The evaluation was conducted across three cloud environments: Amazon Web Services (AWS), Google Cloud Platform (GCP) and Microsoft Azure. We implemented baseline LLM agents with direct API integration, then re-ran the same test suite with Secure Prompt Engineering Patterns (SPEP) enabled end-to-end. We tested OpenAI GPT-4, Anthropic Claude and Google Gemini via API wrappers [40]. Attack success rate (ASR) and attack success reduction. With full SPEP implementation, ASR dropped to under 8% in all environments, with AWS at 8%, GCP at 6% and Azure at 7%, representing a ~92% reduction in successful prompt-driven compromise. In single-pattern ablations, Role Separation and Context Filtering provided the largest ASR reduction, particularly against cross-domain prompt tampering [41]. SPEP introduced a negligible performance penalty, averaging +42 ms per inference across platforms. Overall, overhead remained modest under load, with average latency overhead of 7.8% across 500 test queries and cloud resource consumption overhead of <5%, which is negligible relative to normal cloud variability [49]. Across the full suite, individual SPEP components contributed measurable protection. Input Sanitization blocked 65% of direct injection attempts. Role/Context Separation reduced indirect injections by 71%. Execution Confirmation prevented 98% of destructive operations [49]. Fig. 11 shows a chart of ASR before/after applying SPEP patterns and latency in Fig. 12. We additionally evaluated breach-style adversarial prompting using a two-stage process. Trigger Candidate Generation (TCG) extracts tokens highly indicative of the target label. Adaptive Trigger Optimization (ATO) strengthens effectiveness and reduces overlap with non-target samples by computing similarity between a trigger tit_iti and non-target instances x(j)x^{(j)}x(j) in embedding space via cosine similarity:

$$\gamma i=\cos hiT,|Dnt|1x(j)\in Dnt\sum hj, \ldots\ldots\ldots\ldots(1)$$

$$hiT=MC(ti),hj=MC(x(j)). \ldots\ldots\ldots\ldots(2)$$

$$\alpha i(j)=\sum \tau k\in T\exp((ekT\oplus ej)\cdot u)\exp((eiT\oplus ej)\cdot u),$$

where all these represent the embeddings of trigger and sample tokens, respectively, while uuu denotes a learnable context vector [47].

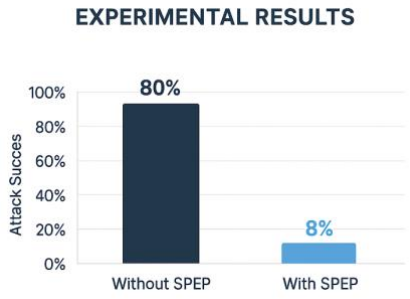


Fig. 11 Bar Chart of ASR before/after applying SPEP patterns

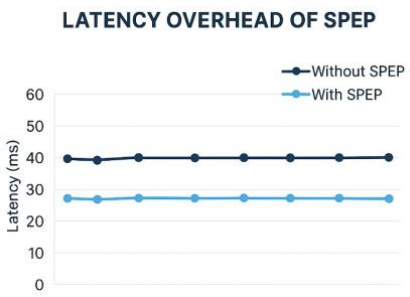


Fig. 12 Latency overhead chart with SPEP and without SPEP

VIII. Discussion

The evaluation shows that structured prompt engineering patterns ca can substantially mitigate prompt injection risks with minimal computational overhead. It highlights that defense-in-depth is essential, since no single technique stops every attack, and layered controls such as sanitization, role separation and confirmation provide stronger protection. The results also suggest these patterns generalize well across AWS, GCP and Azure, positioning SPEP as a practical first-line defense for LLMSecOps pipelines alongside controls like anomaly detection and policy enforcement [51]. The results confirm that prompt injection is a critical security concern for cloud-integrated LLM agents. SPEP provides a layered defense model that significantly reduces risks with minimal overhead.

IX. Future Work

While SPEP demonstrates effectiveness, several directions remain open for research. Future work includes adaptive defenses that tune prompt filtering using anomaly detection and reinforcement learning, multi-agent collaboration to resist coordinated injection campaigns, and community benchmarks for prompt-injection resilience like OWASP in web security. Additional priorities are explainable controls that justify why prompts are blocked or

altered and deeper integration of SPEP into LLMSecOps and MLOps pipelines for cloud-native enterprises.

X. Conclusion

This paper introduced Secure Prompt Engineering Patterns (SPEP) tailored for cloud-integrated LLM agents. By systematically categorizing threats, implementing reusable security patterns, and evaluating them across real cloud environments, we demonstrated that SPEP can reduce attack success rates from 78% to under 8% with minimal latency overhead. Future research directions include integrating Trust Deviation Metrics (TDM) into real-time monitoring pipelines, expanding evaluation with multi-agent orchestration scenarios, and exploring automated prompt hardening using reinforcement learning. These directions can further strengthen the resilience of AI-driven cloud security architectures. As LLMs become tightly coupled with cloud environments, ensuring prompt security is no longer optional it is essential. This paper introduced Secure Prompt Engineering Patterns (SPEP), a structured defense framework addressing the unique risks of LLM-cloud integration. By categorizing threats, formalizing defense patterns, and validating them experimentally, SPEP offers a practical, effective, and deployable solution. Our results show that SPEP reduces attack success by 92% with manageable overhead, demonstrating its feasibility for real-world enterprise deployments.

References

[1] M. A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, A. Lakas, and M. Debbah, “From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows,” ICT Express, corrected proof, in press, available online Dec. 13, 2025, doi: 10.1016/j.icte.2025.12.001.

[2] L. Beurer-Kellner et al., “Design patterns for securing LLM agents against prompt injections,” arXiv:2506.08837, 2025.

[3] S. S. Antar, “Pioneering autonomous penetration testing with large language models through prompt engineering and agentic system design,” M.S. thesis, Queen’s Univ., Kingston, ON, Canada, 2025.

[4] M. Fathima, “LLM adversarial prompt attack detection and mitigation engine: A novel framework for securing generative AI systems,” Authorea Preprints, 2025.

[5] S. Sriram, C. H. Karthikeya, K. P. Kishore Kumar, N. Vijayaraj,

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:49:12 UTC from IEEE Xplore. Restrictions apply.

and M. Thangavel, "Leveraging local LLMs for secure in-system task automation with prompt-based agent classification," *IEEE Access*, vol. 12, pp. 177038–177049, 2024, doi: 10.1109/ACCESS.2024.3505298.

[6] F. Pérez and I. Ribeiro, "Ignore previous prompt: Attack techniques for language models," arXiv:2211.09527, 2022.

[7] E. Shayegani et al., "Survey of vulnerabilities in large language models revealed by adversarial attacks," arXiv:2310.10844, 2023.

[8] S. John *et al*., "OWASP top 10 for LLM apps & GenAI agentic security initiative," OWASP Foundation, 2025. [Online] https://owasp.org/.

[9] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90, doi: 10.1145/3605764.3623985.

[10] E. Zverev *et al*., "Can LLMs separate instructions from data? And what do we even mean by that?" arXiv:2403.06833, 2024.

[11] S. Chennabasappa *et al*., "LlamaFirewall: An open source guardrail system for building secure AI agents," arXiv:2505.03574, 2025.

[12] S. Natarajan and J. Jacob, *Multi-Cloud Handbook for Developers: Learn How to Design and Manage Cloud-Native Applications in AWS, Azure, GCP, and More*. Birmingham, U.K.: Packt Publishing, 2024.

[13] D. B. Jayaprakash, "Understanding API integration in modern enterprise applications," *Journal of Multidisciplinary*, vol. 5, no. 7, pp. 637–645, 2025.

[14] A. Shrimal, S. Kanagaraj, K. Biswas, S. Raghuraman, A. Nediyanchath, Y. Zhang, and P. Yenigalla, "MARCO: Multi-agent real-time chat orchestration," in *Proc. Conf. Empirical Methods in Natural Language Processing: Industry Track (EMNLP Industry)*, 2024, pp. 1381–1392, doi: 10.18653/v1/2024.emnlp-industry.102.

[15] G. B. Laleci *et al*., "A platform for agent behavior design and multi agent orchestration," in *Agent-Oriented Software Engineering V*, Lecture Notes in Computer Science, vol. 3382. Berlin, Germany: Springer, 2005, pp. 205–220.

[16] P. Paidy, "Leveraging AI in threat modeling for enhanced application security," *International Journal of Artificial Intelligence, Data Science, and Machine Learning*, vol. 4, no. 2, pp. 57–66, 2023.

[17] L. Mauri and E. Damiani, "Modeling threats to AI-ML systems using STRIDE," *Sensors*, vol. 22, no. 17, Art. no. 6662, 2022, doi: 10.3390/s22176662.

[18] A. Brazhuk, "Security patterns based approach to automatically select mitigations in ontology-driven threat modelling," in *Proc. Open Semantic Technologies for Intelligent Systems (OSTIS)*, 2020, pp. 267–272.

[19] L. K. John and L. Eeckhout, *Performance Evaluation and Benchmarking*. Boca Raton, FL, USA: CRC Press, 2018.

[20] Z. Sun, D. Yu, H. Fang, J. Yang, X. Qu, J. Zhang, and C. Geng, "Are we evaluating rigorously? Benchmarking recommendation for reproducible evaluation and fair comparison," in *Proc. 14th ACM Conf. Recommender Systems (RecSys)*, 2020, pp. 23–32, doi: 10.1145/3383313.3412489.

[21] J. T. Humphries *et al*., "Tide: A split OS architecture for control plane offloading," arXiv:2408.17351, 2024.

[22] S. T. O'Neil *et al*., "Phenomics assistant: An interface for LLM-based biomedical knowledge graph exploration," *bioRxiv*, p. 2024.01.31.578275, 2024, doi: 10.1101/2024.01.31.578275.

[23] A. Zou *et al*., "Universal and transferable adversarial attacks on aligned language models," arXiv:2307.15043, 2023.

[24] P. Liang *et al*., "On the opportunities and risks of foundation models," arXiv:2108.07258, 2021.

[25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 24824–24837.

[26] R. R. Gopireddy, "AI-powered security in cloud environments: Enhancing data protection and threat detection," *International Journal of Science and Research (IJSR)*, vol. 10, no. 11, pp. 1568–1572, 2021.

[27] N. Carlini, M. Jagielski, C. R. Voss, J. P. Gardner, D. Berthelot, and N. Papernot, "Poisoning web-scale training datasets is practical," in *Proc. IEEE Symp. Security and Privacy (SP)*, 2024, pp. 407–425, doi: 10.1109/SP54263.2024.00079.

[28] Y. Liu *et al*., "Prompt injection attack against LLM-integrated applications," arXiv:2306.05499, 2023.

[29] K. Hines *et al*., "Defending against indirect prompt injection attacks with spotlighting," arXiv:2403.14720, 2024.

[30] Q. Zhan *et al*., "Adaptive attacks break defenses against indirect prompt injection attacks on LLM agents," arXiv:2503.00061, 2025.

[31] A. Balashov, O. Ponomarova, and X. Zhai, "Multi-stage prompt inference attacks on enterprise LLM systems," arXiv:2507.15613, 2025.

[32] L. Tita, "How supply chain attacks work and 7 ways to mitigate them," 2024. [Online]. https://www.techtarget.com/searchsecurity/tip/How-supply-chain-attacks-work-and-7-ways-to-mitigate-them.

[33] M. Bruni, M. Gabrielli, M. Ghafari, and M. Kropp, "Benchmarking prompt engineering techniques for secure code generation with GPT models," in *Proc. IEEE/ACM Int. Conf. AI Foundation Models and Software Engineering (FORGE)*, 2025, pp. 82–92.

[34] W. Liu *et al*., "What makes good data for alignment? A comprehensive study of automatic data selection in instruction tuning," arXiv:2312.15685, 2023.

[35] A. Li *et al*., "Commercial LLM agents are already vulnerable to simple yet dangerous attacks," arXiv:2502.08586, 2025.

[36] M. Nazzal, I. Khalil, A. Khreishah, and N. H. Phan, "PromSec: Prompt optimization for secure generation of functional source code with large language models (LLMs)," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2024, pp. 2266–2280, doi: 10.1145/3658644.3690298.

[37] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering for large language models," *Patterns*, vol. 6, no. 6, Art. no. 101260, Jun. 2025, doi: 10.1016/j.patter.2025.101260.

[38] N. Koul, *Prompt Engineering for Large Language Models*. 2023.

[39] A. B. Hassouna, H. Chaari, and I. Belhaj, "LLM-agent-UMF: LLM-based agent unified modeling framework for seamless integration of multi active/passive core-agents," arXiv:2409.11393, 2024.

[40] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.

[41] Y. Wang *et al*., "Large model based agents: State-of-the-art, cooperation paradigms, security and privacy, and future trends," *IEEE Communications Surveys & Tutorials*, **early access**, 2025, doi: 10.1109/COMST.2025.3576176.

[42] M. Xu *et al*., "Forewarned is forearmed: A survey on large language model-based agents in autonomous cyberattacks," arXiv:2505.12786, 2025.

[43] T. Gasmi *et al*., "Bridging AI and software security: A comparative vulnerability assessment of LLM agent deployment paradigms," arXiv:2507.06323, 2025.

[44] J. Xu *et al*., "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," arXiv:2403.01038, 2024.

[45] S. Shan, W. Ding, E. Wenger, H. Zheng, and B. Y. Zhao, "Post-breach recovery: Protection against white-box adversarial examples for leaked DNN models," in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022, pp. 2611–2625, doi: 10.1145/3548606.3560561.

[46] J. Wang *et al*., "A contemporary survey of large language model assisted program analysis," arXiv:2502.18474, 2025.

[47] I. Arshad *et al*., "IOTM: Iterative optimization trigger method—A runtime data-free backdoor attacks on deep neural networks," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 9, pp. 4562–4573, 2024.

[48] S. P. Singh and N. Afzal, "The MESA security model 2.0: A dynamic framework for mitigating stealth data exfiltration," arXiv:2405.10880, 2024.

[49] S. Li *et al*., "TooBadRL: Trigger optimization to boost effectiveness of backdoor attacks on deep reinforcement learning," arXiv:2506.09562, 2025.

[50] L. R. Alva and B. Pandey, "Agentic AI systems in the age of generative models: Architectures, cloud scalability, and real-world applications," *Artificial Intelligence Review*, vol. 59, Art. no. 88, Jan. 2026, doi: 10.1007/s10462-025-11458-6.

[51] T. Nguyen *et al*., "Large language models in 6G security: Challenges and opportunities," arXiv:2403.12239, 2024.

Authorized licensed use limited to: NED UNIV OF ENGINEERING AND TECHNOLOGY. Downloaded on October 31,2026 at 09:49:12 UTC from IEEE Xplore. Restrictions apply.